sesa lab
SOFTWARE ENGINEERING
SALERNO

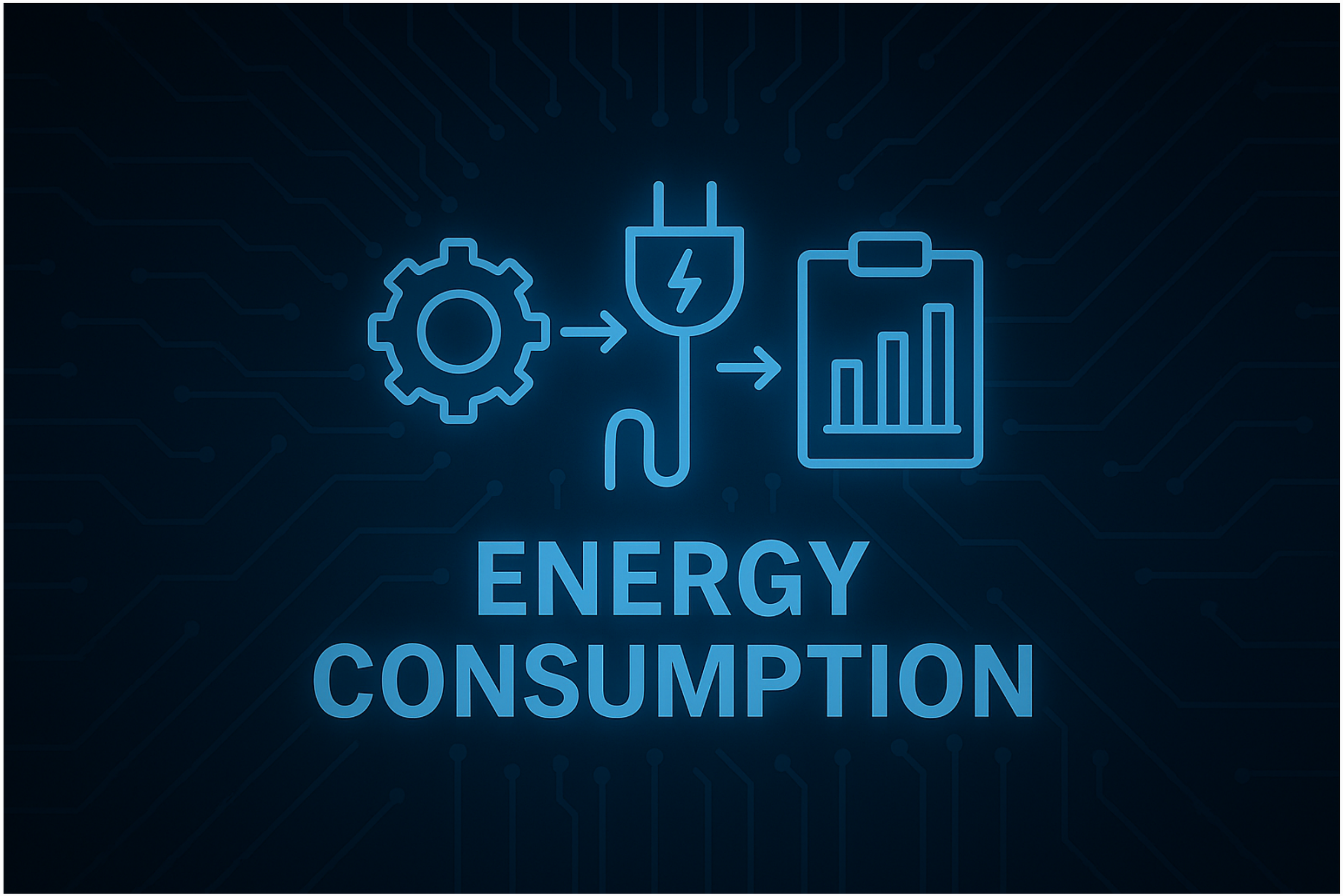
sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Measuring Energy Consumption



Slides courtesy of Dario Di Nucci | 2026 - 2027

How to Measure Energy Consumption?



How to Measure Energy Consumption?

Energy tests are not deterministic; running them multiple times yields different outputs.

Several factors must be minimized before using a measurement tool.

First, you need to reduce background activities:

- Close all running applications.
- Terminate all unnecessary background services.
- Turn off notifications.
- Turn off or unplug all unnecessary hardware, including Wi-Fi, if you do not need the internet.
- If you need internet, use a wired Ethernet connection.
- Switch off auto-brightness, keep the screen brightness fixed.

How to Measure Energy Consumption?

The environment is more important than in functional testing!

Hardware temperature impacts your results.

The higher the temperature, the greater the energy consumption.

Run some tests without measuring the temperature to warm up the machine.

Conduct all tests in the same room with a stable temperature.

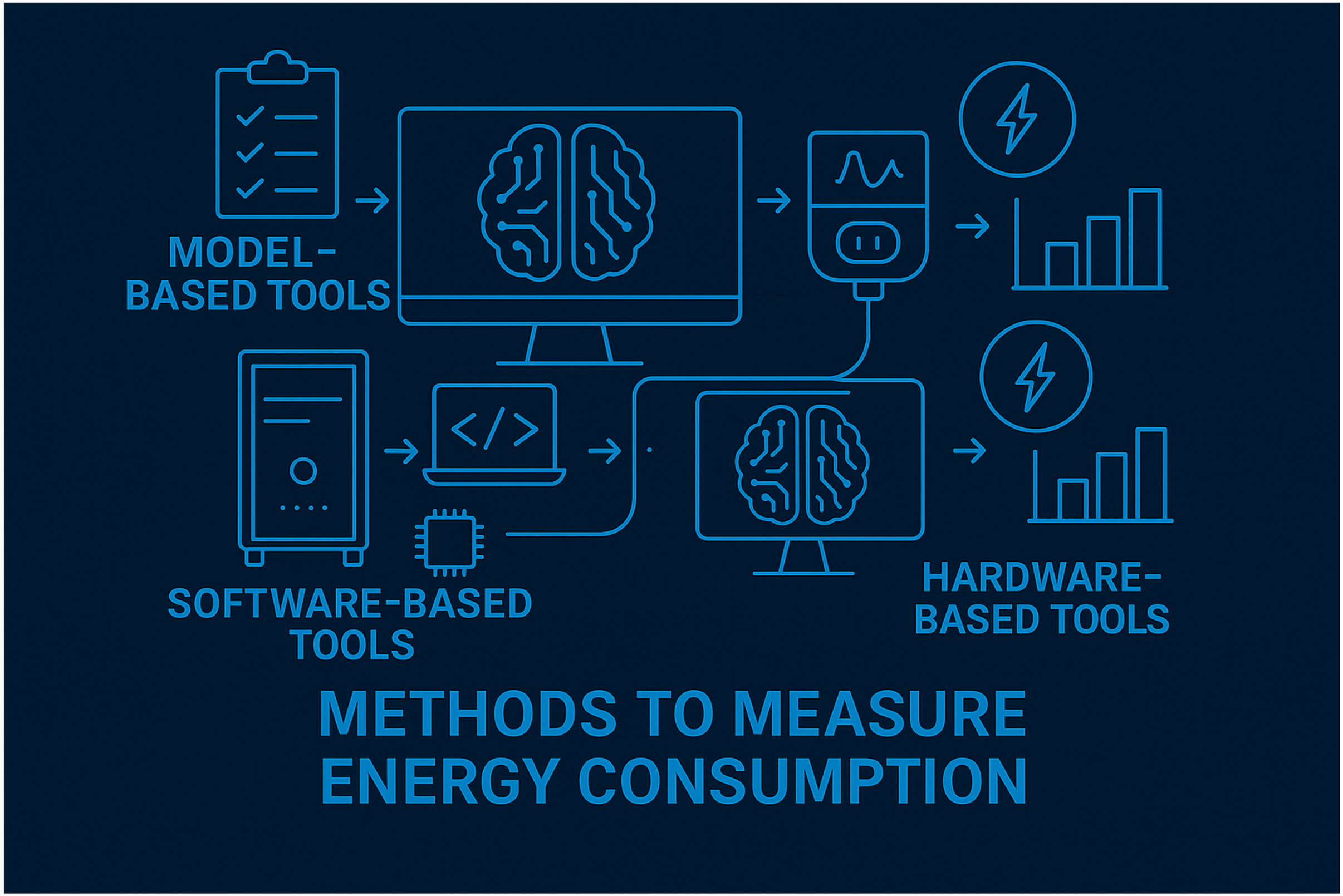
Alternatively, collect data about the temperature to exclude measurements taken under different environmental conditions.

Since energy tests are not deterministic, you need to run them multiple times.

Make sure to pause between runs to prevent the results of one run from affecting the next.

Automatically shuffling clusters of tests can also be helpful.

How to Measure Energy Consumption?



Harman et al. Achievements, open problems and challenges for search based software testing.
2015. IEEE International Conference on Software Testing

Model-based Tools



MODEL-BASED TOOLS TO MEASURE ENERGY CONSUMPTION

Model-based Tools: PowerBooter and PowerTutor

PowerBooter is a technique for automated power model construction that relies on battery voltage sensors and battery discharge knowledge. It does not require external power meters.

PowerTutor uses the model provided by PowerBooter for generating online power estimation.



Figure 3: Wi-Fi interface power states.

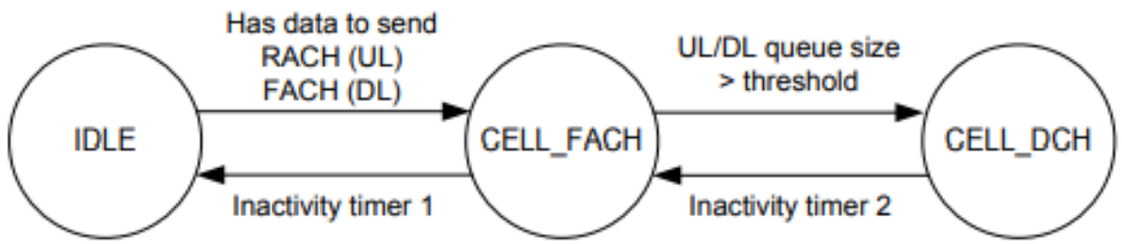


Figure 4: 3G interface power states.

Zhang, Lide, et al. "Accurate online power estimation and automatic battery behavior based power model generation for smartphones." *Proceedings of the eighth IEEE/ACM/IFIP international conference on Hardware/software codesign and system synthesis*. 2010.

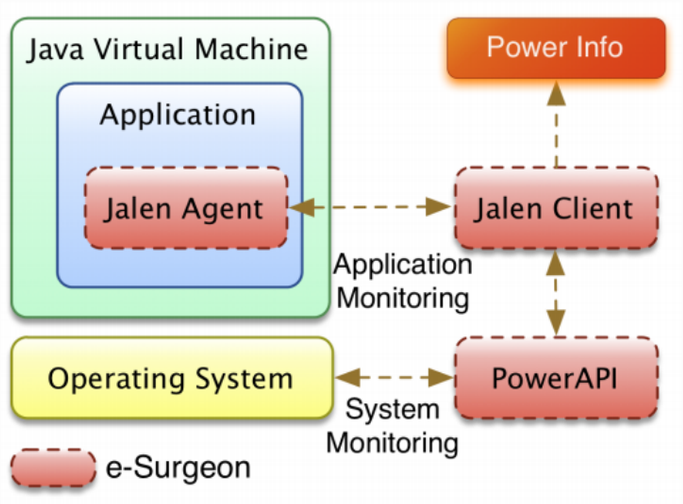
Model-based Tools: e-Surgeon

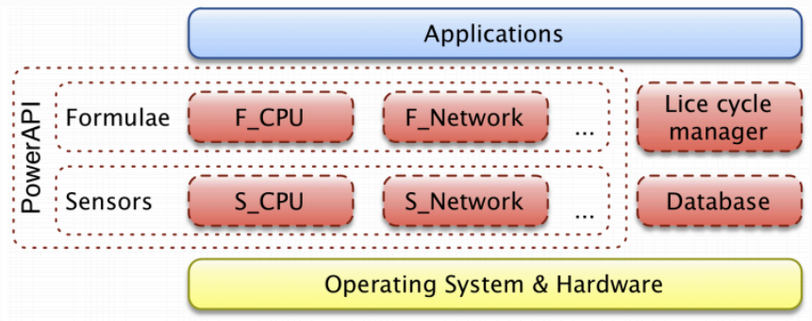
e-Surgeon is composed of two distinct but complementary parts:

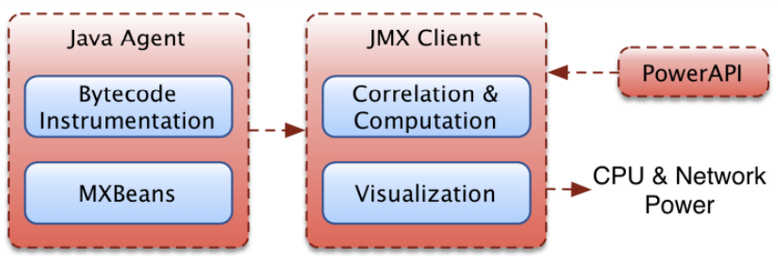
- PowerAPI, a system-level power monitoring environment;
- Jalen, a software-level application profiling environment.

These two parts work together to provide accurate runtime energy information at the application level (threads and methods levels).

The consumption of a software unit is computed as $P_{software} = P_{comp} + P_{com}$ where P_{comp} is a component consumption (e.g., CPU) and P_{com} is the communication consumption (e.g., the network).







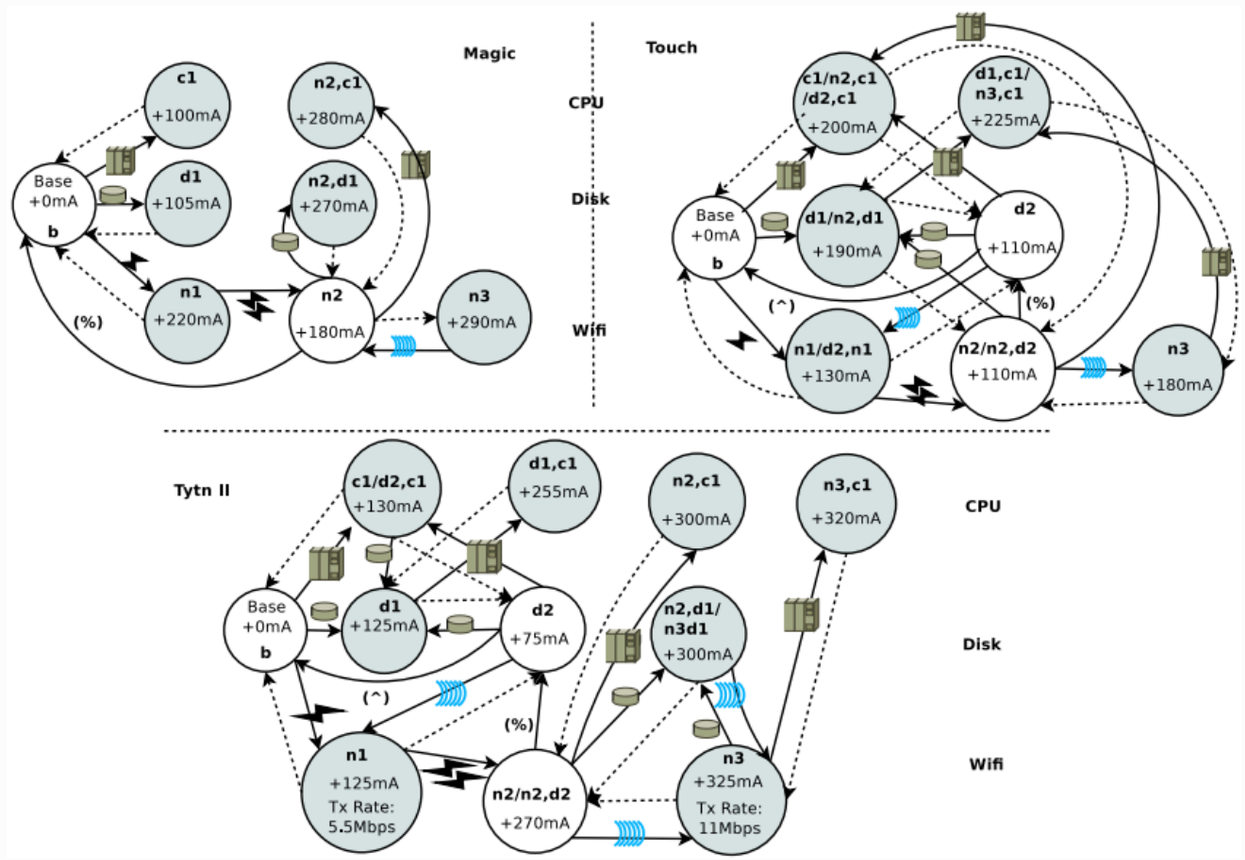
Noureddine, Adel, et al. "Runtime monitoring of software energy hotspots."
2012 Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering. IEEE, 2012.

Model-based Tools: eProf

Abhinav et al. developed a Finite State Machine for each hardware component.
They used a power monitor to measure the energy consumption of each transaction.

They bypassed the smartphone battery, connecting the power terminals of the phone to the power supply passing through the power monitor, with a sampling rate of 200 microseconds.

eProf, the tool implementing the model, manually annotates every subroutine in the source code to log entry and exit points. It also enables tracing each system call to the caller subroutine.



Abhinav et al. ”Where is the energy spent inside my app? Fine Grained Energy Accounting on Smartphones with Eprof.”
Proceedings of the 7th ACM european conference on Computer Systems. 2012.

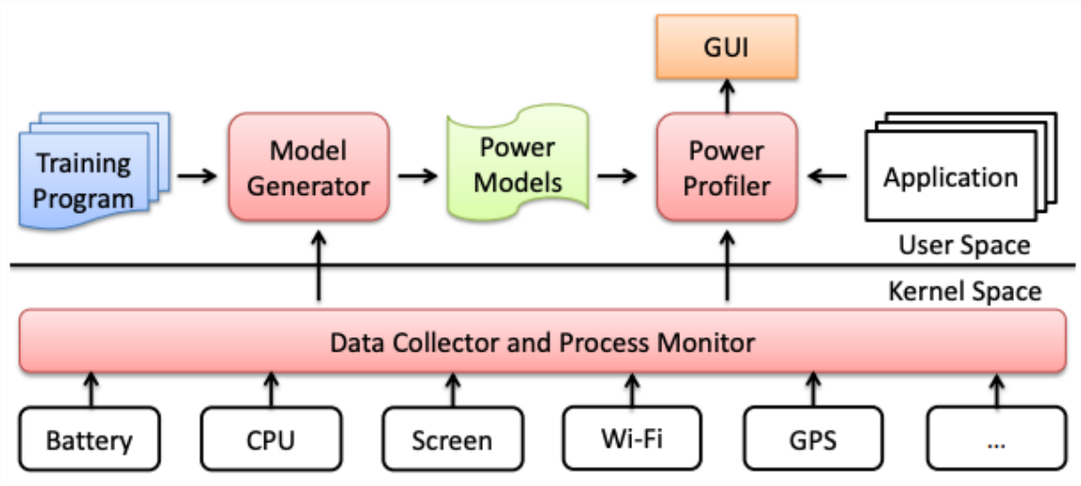
Model-based Tools: V-edge

V-edge automatically generates component-level power models by collecting:

- voltage readings from the battery interface;
- usage information of each hardware component (e.g., CPU, screen, Wi-Fi, and GPS);
- process execution and switching information.

The power models are created by running a set of training programs for each component in a controlled way. All training programs of a component run from the same initial state to ensure their measured V-edge values are consistent.

Power estimation is done by tracking hardware resource usage and applying the generated models.



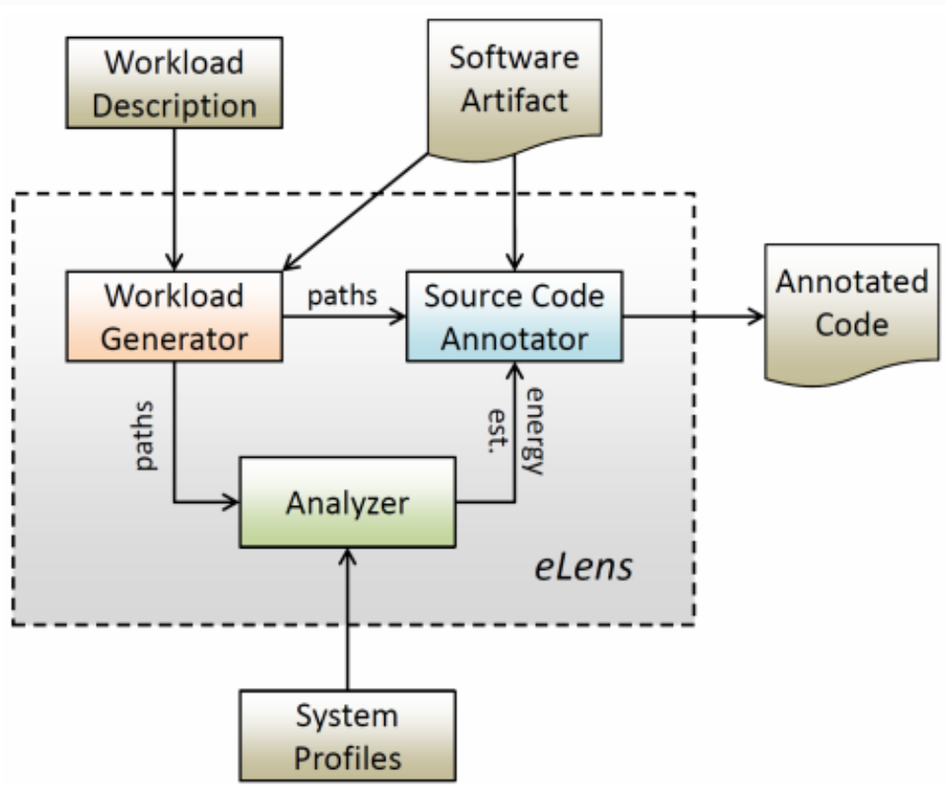
Xu, Fengyuan, et al. "V-edge: Fast self-constructive power modeling of smartphones based on battery voltage dynamics." 10th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 13). 2013.

Model-based Tools: eLens

eLens provides a more fine-grained estimation of energy consumption at the method, path, or line-of-source level. It relies on a combination of program analysis and energy modeling. It produces visual feedback to help developers better understand the application behavior.

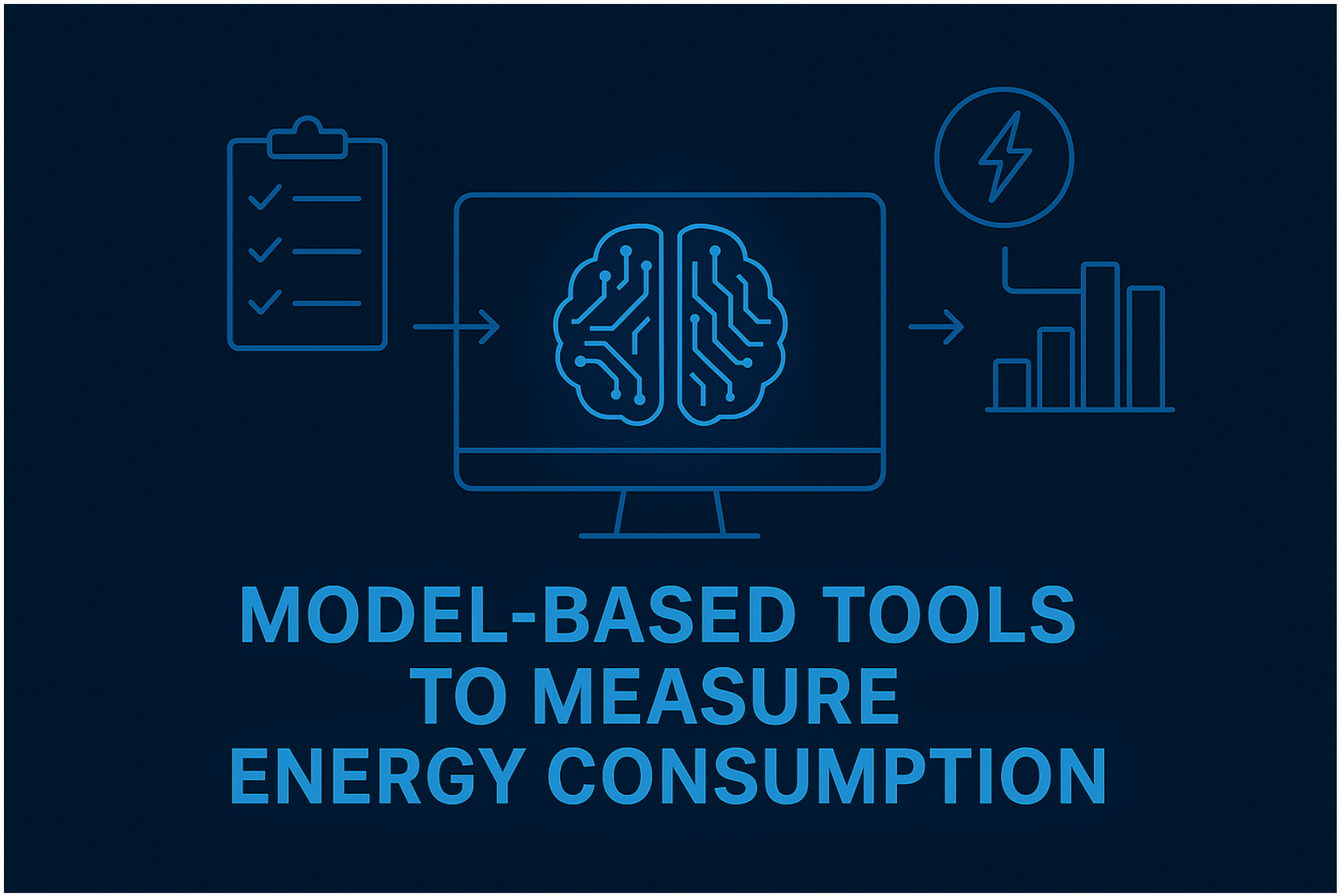
Most components (e.g., CPU, network, and GPS) have multiple power states:

- Modern smartphone CPUs can operate at different frequencies, which consume different amounts of energy.
- The network may be idle, transmitting/receiving, or have selectively enabled multiple antennas.
- The GPS may be on or off.



Xu, Fengyuan, et al. "V-edge: Fast self-constructive power modeling of smartphones based on battery voltage dynamics." 10th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 13). 2013.

Model-based Tools

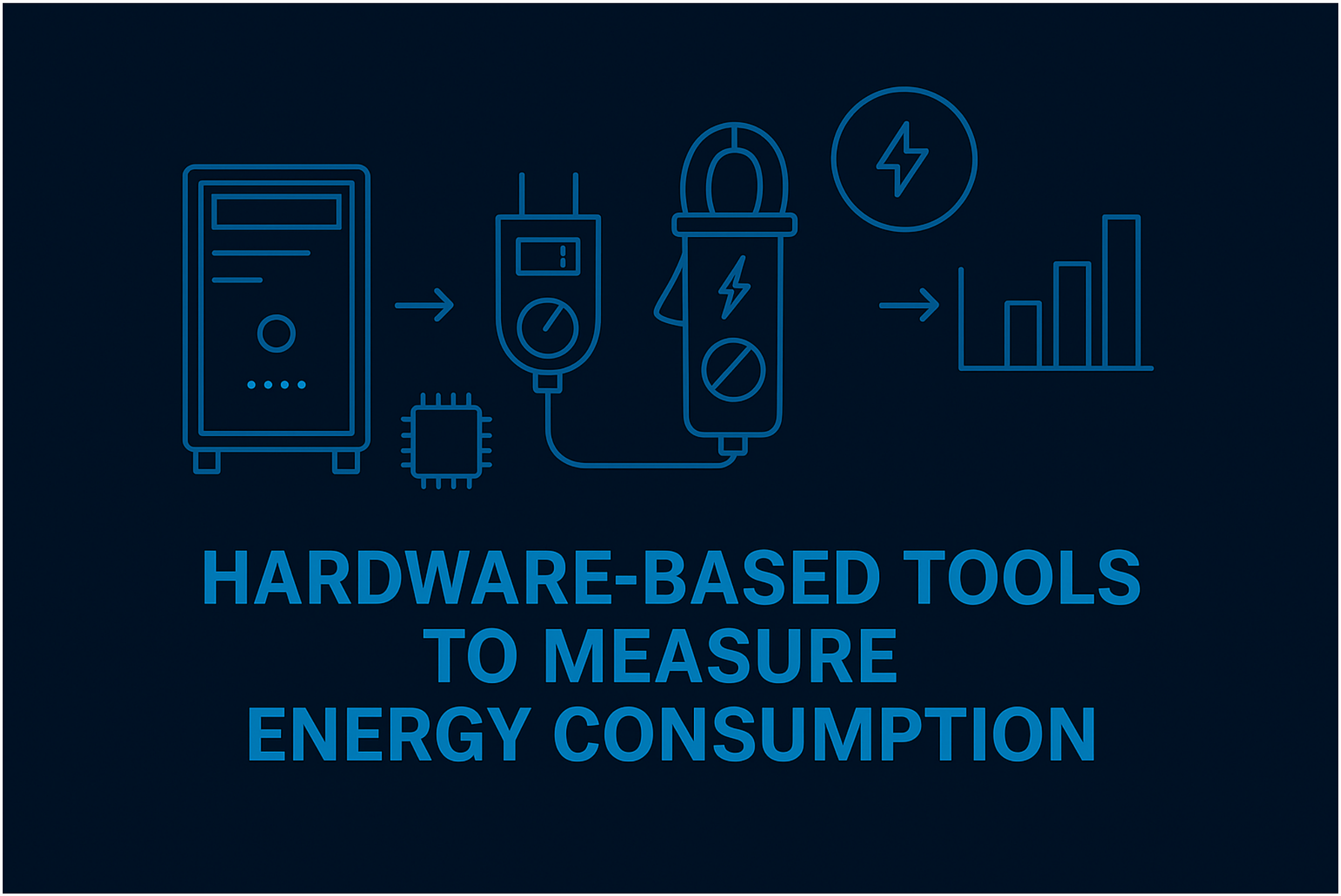


+ Not require specialized HW

- Less precise

- Need careful parameter calibration

Hardware-based Tools

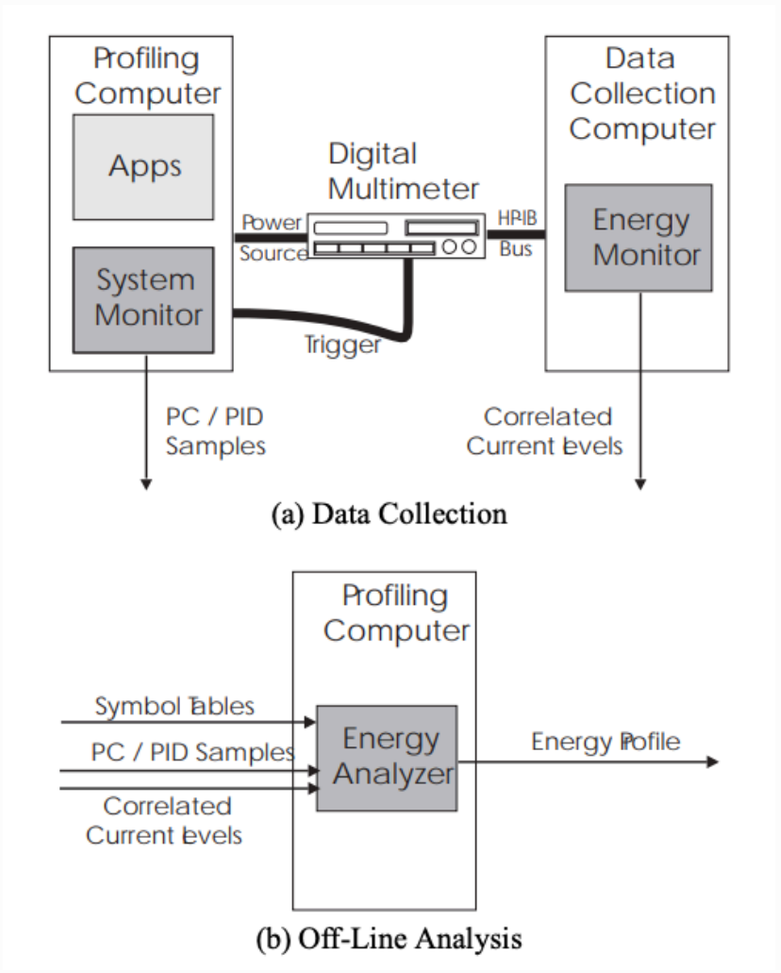


Hardware-based Tools: PowerScope

PowerScope utilizes statistical sampling to profile the energy usage of a computer system, employing a two-stage process to minimize the overhead associated.

During the data collection stage, the tool samples both the power consumption and the system activity of the profiling computer. This data is used to create the energy profiles.

Because the analysis is performed offline, PowerScope has no profiling overhead.



Flinn, Jason, and Mahadev Satyanarayanan. "Powerscope: A tool for profiling the energy usage of mobile applications." Proceedings WMCSA'99. Second IEEE Workshop on Mobile Computing Systems and Applications. IEEE, 1999.

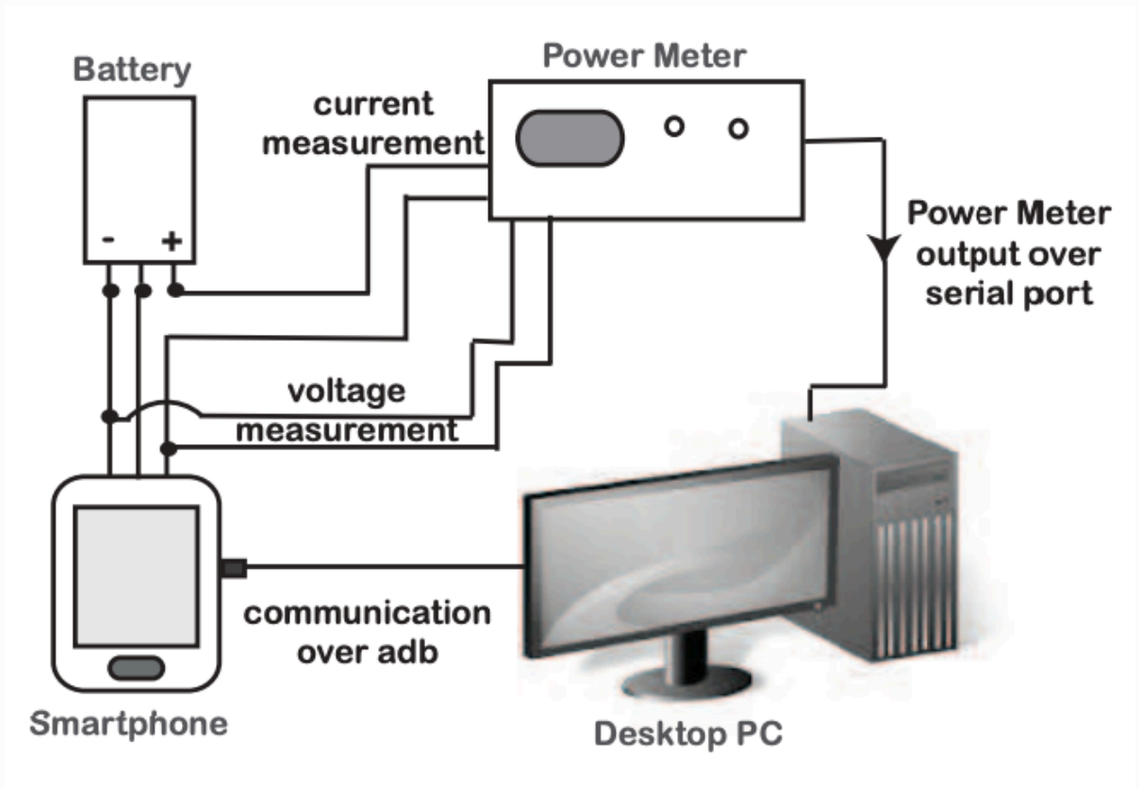
Hardware-based Tools: Banerjee et al.

The framework proposed by Banerjee et al. runs on a PC that also functions as the global clock.

All power meter measurements and smartphone usage data are stored on the PC.

Each reading is timestamped automatically. Since the timestamps are generated by a single clock (the desktop PC's clock), they can be used to synchronize data from the power meter and the smartphone.

Android Debug Bridge is used to communicate with the smartphone. This communication involves sending event traces to the smartphone and recording usage data.



Banerjee, Abhijeet, et al. "Detecting energy bugs and hotspots in mobile apps."
Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. 2014.

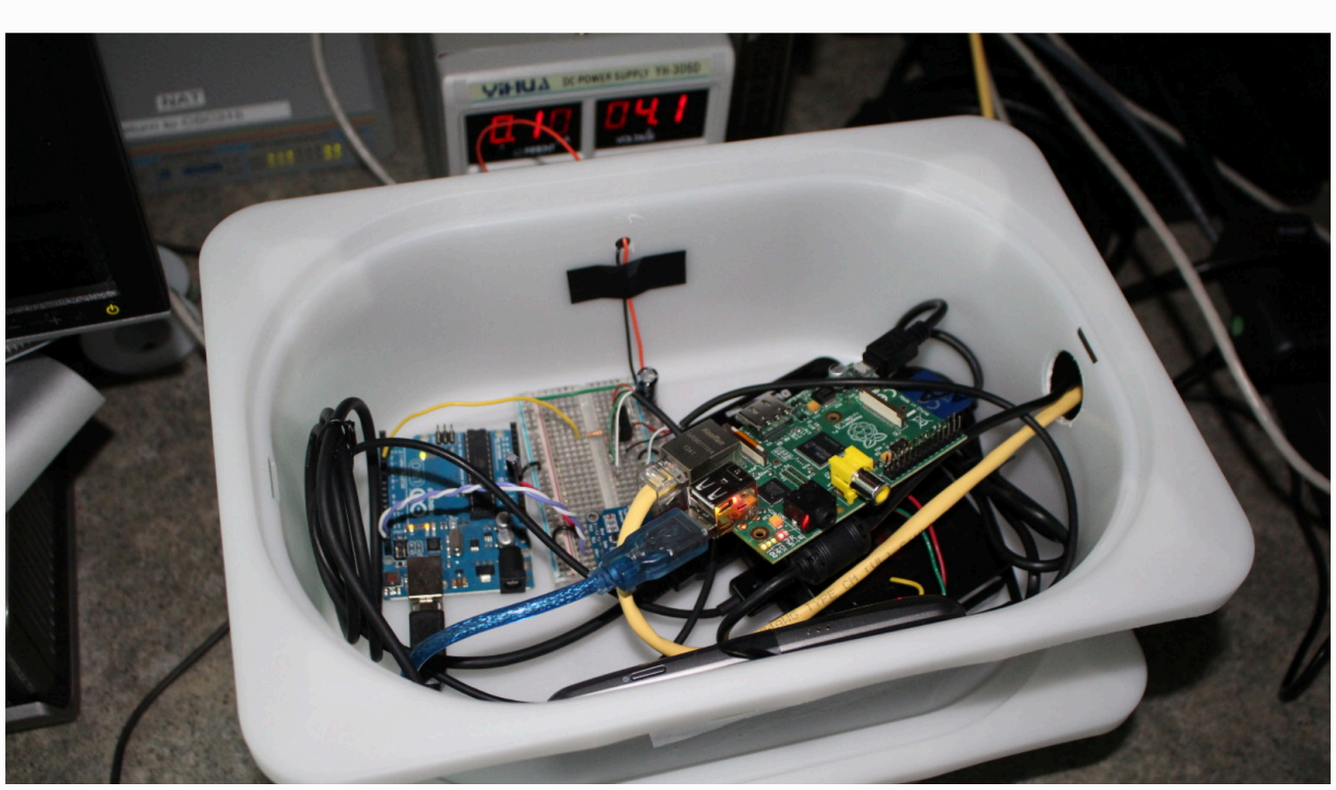
Hardware-based Tools: Green Miner

Green Miner features a client-server setup.

The server features an Arduino and a breadboard equipped with an INA219 chip for measuring energy usage. The client side includes a Raspberry Pi running the Green Miner client, a USB hub, and a Galaxy Nexus phone connected to a high-current 4.1V DC power supply.

The Raspberry Pi handles data collection, uploads, and testing on the Android device.

Batteries cause unstable measurements due to large voltage fluctuations and internal resistance, so they are removed from the phone, and wires are soldered directly to their contacts.



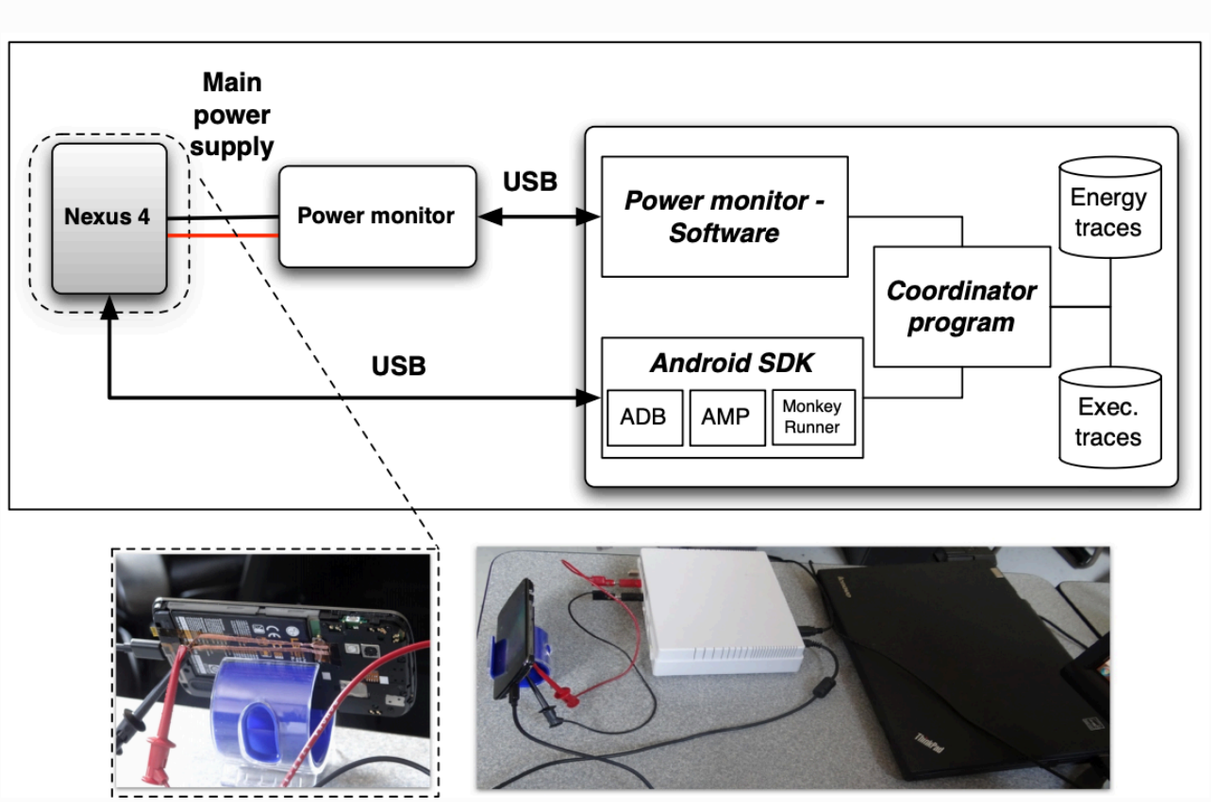
Hindle, Abram, et al. "Greenminer: A hardware based mining software repositories software energy consumption framework." Proceedings of the 11th Working Conference on Mining Software Repositories. 2014.

Hardware-based Tools: Linares-Vásquez et al.

Linares-Vásquez et al. used the Android Activity Manager Profiler (AMP), which requires apps to be enabled for debugging, to collect execution traces.

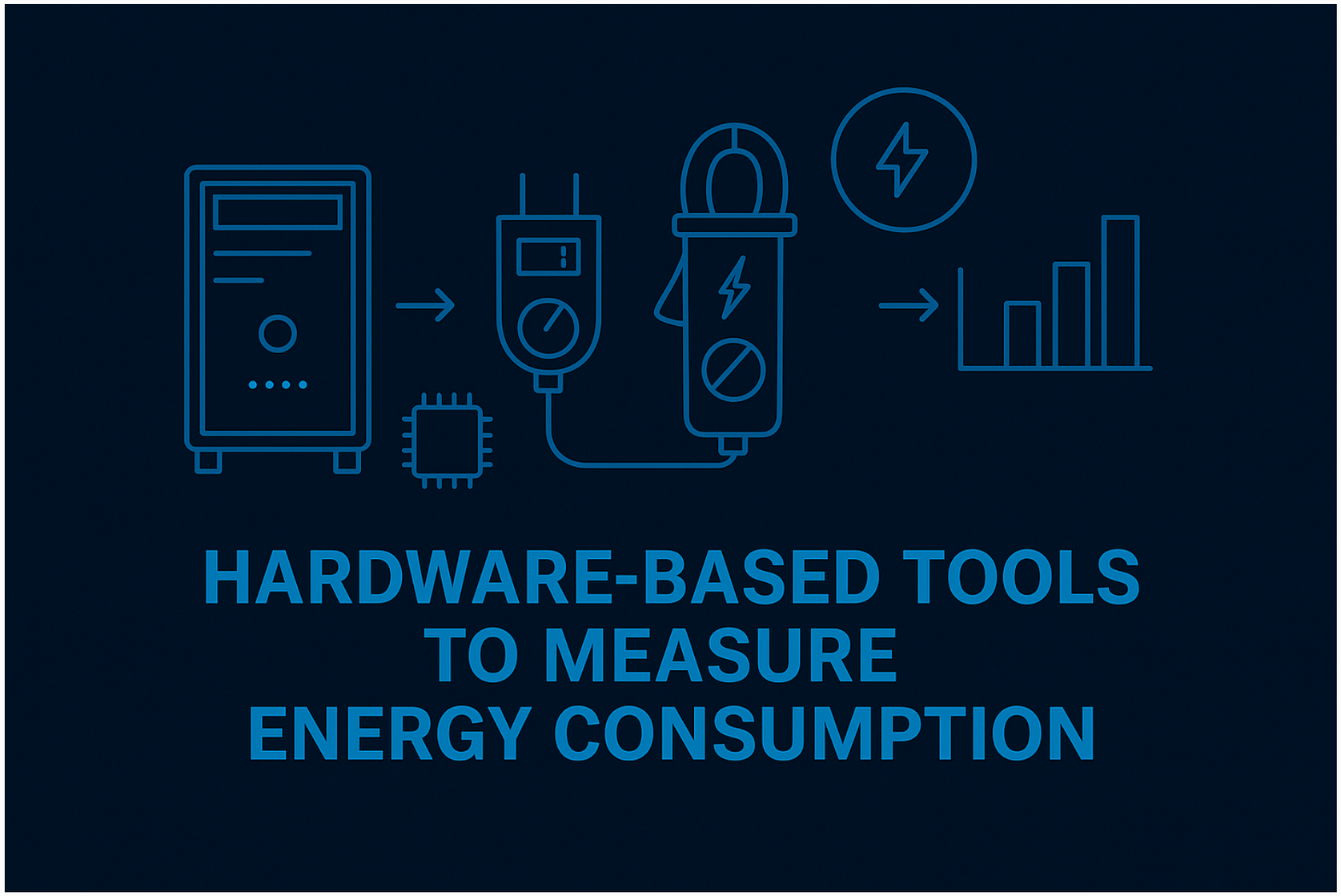
They used the Monsoon power monitor to gather information on power consumption.
USB charging was disabled to prevent any bias in the measurements.

To reduce noisy data, (i) they disabled all unnecessary apps and processes running on the phone to avoid race conditions; (ii) they put the phone in airplane mode to eliminate cell radio power usage and asynchronous events from incoming messages or calls; (iii) they enabled WiFi since some apps need it; and (iv) to prevent energy consumption by sensors, they kept the phone steady.



Linares-Vásquez, Mario, et al. "Mining energy-greedy api usage patterns in android apps: an empirical study." Proceedings of the 11th Working Conference on Mining Software Repositories. 2014.

Hardware-based Tools



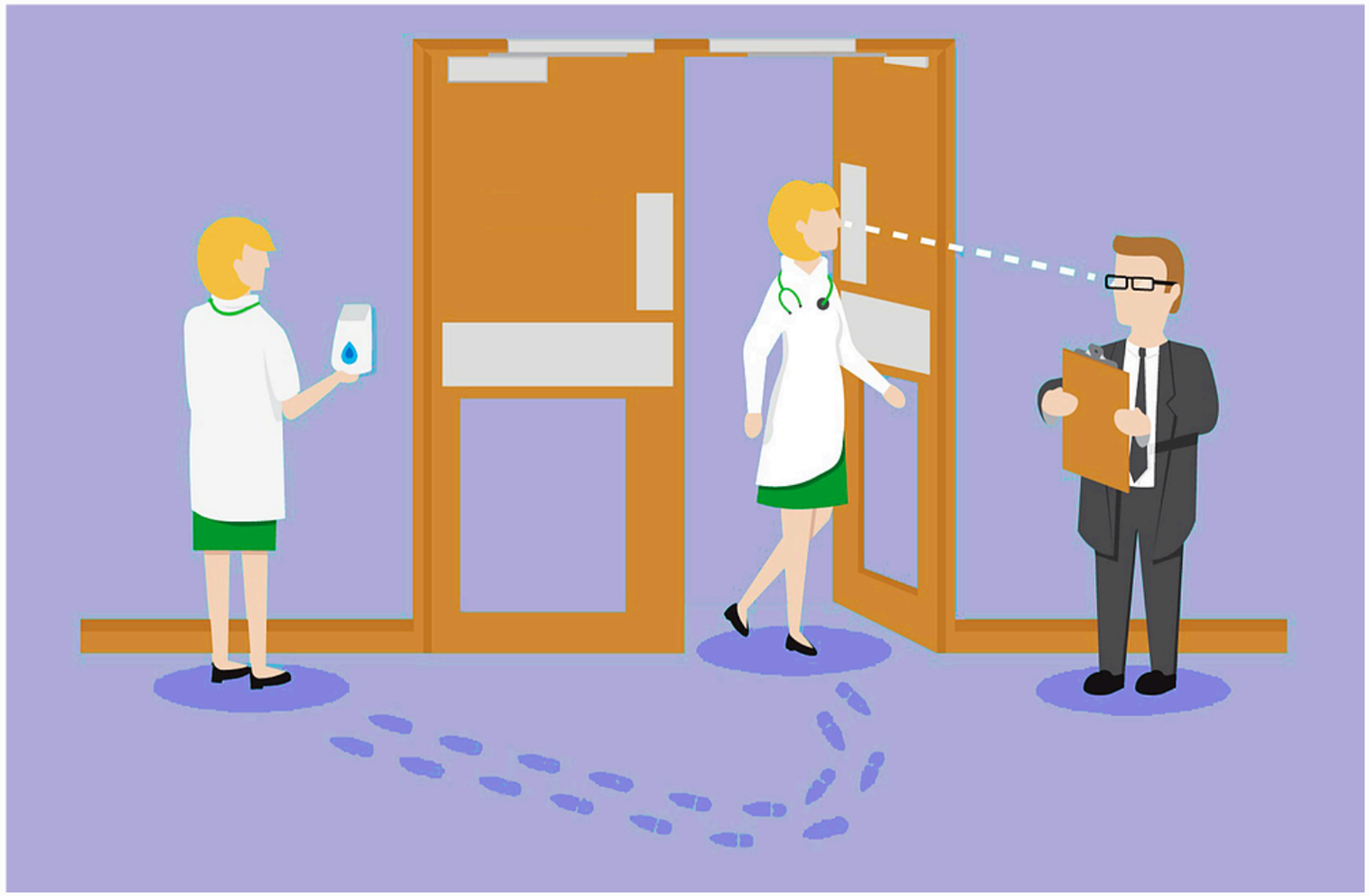
+ Best precision

- Require specialized HW and people

- Hawthorne effect

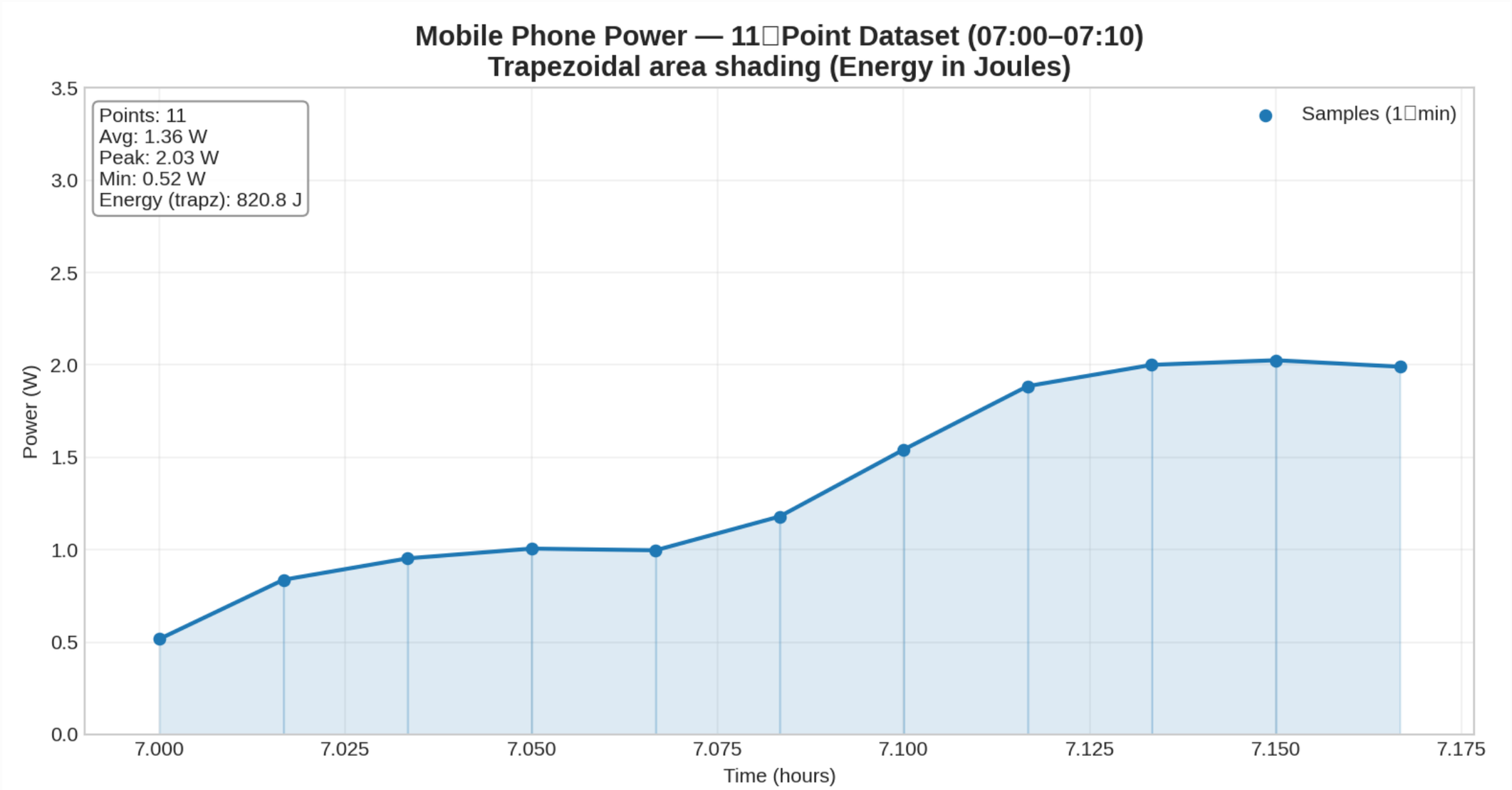
- Sampling issue

Hardware-based Tools: the Hawthorne Effect



The Hawthorne effect (also known as the observer effect) is a phenomenon where individuals alter their behavior in response to being observed.

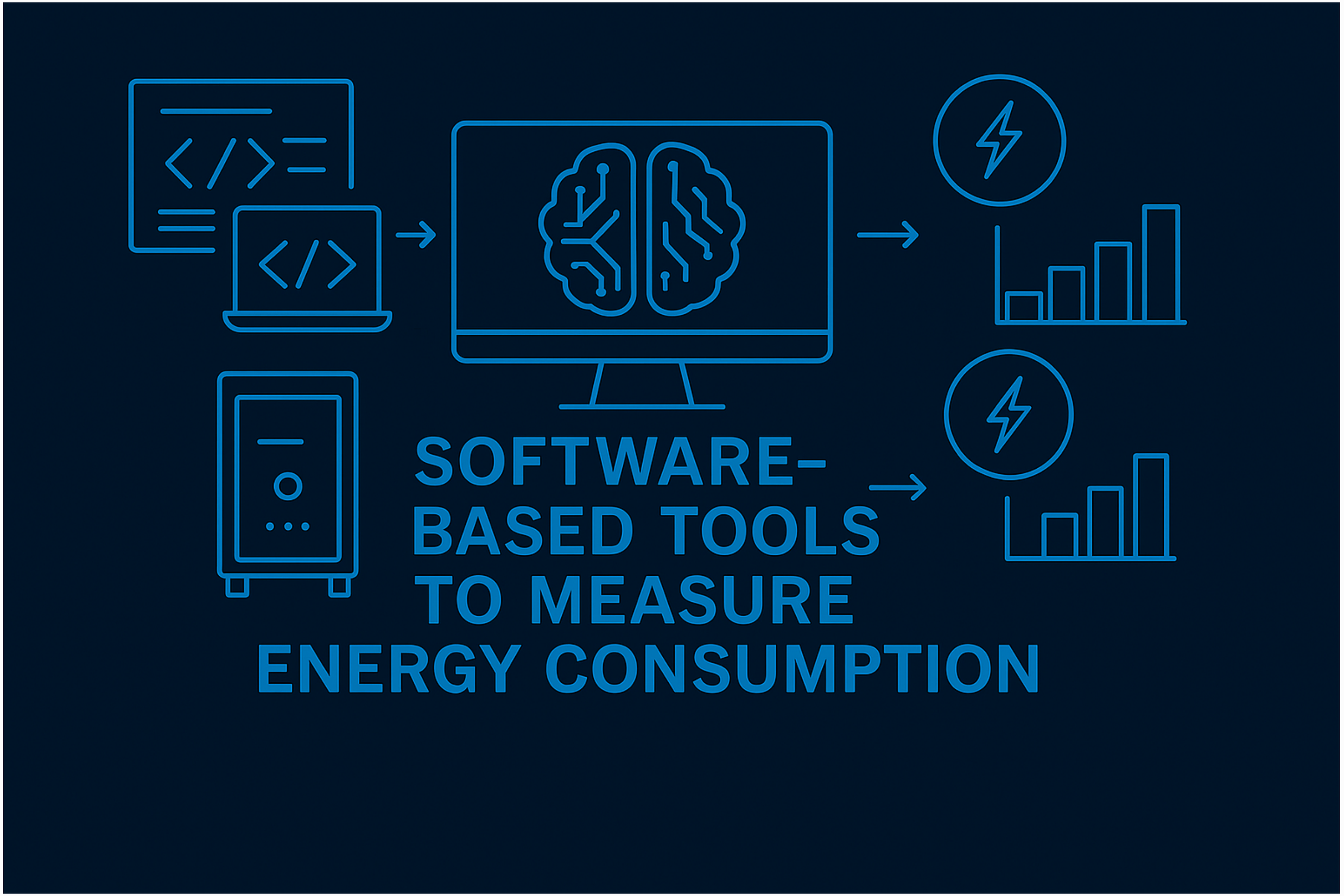
Hardware-based Tools: Energy Sampling



Low-frequency sampling provides a rough estimate, which limits the understanding of detailed features. Sampling at 10 kHz can result in a median error of 8% that may increase to up to 50% for short, rapid methods.

Saborido, Rubén, et al. On the impact of sampling frequency on software energy measurements. No. e1495. PeerJ PrePrints, 2015.

Software-based Tools

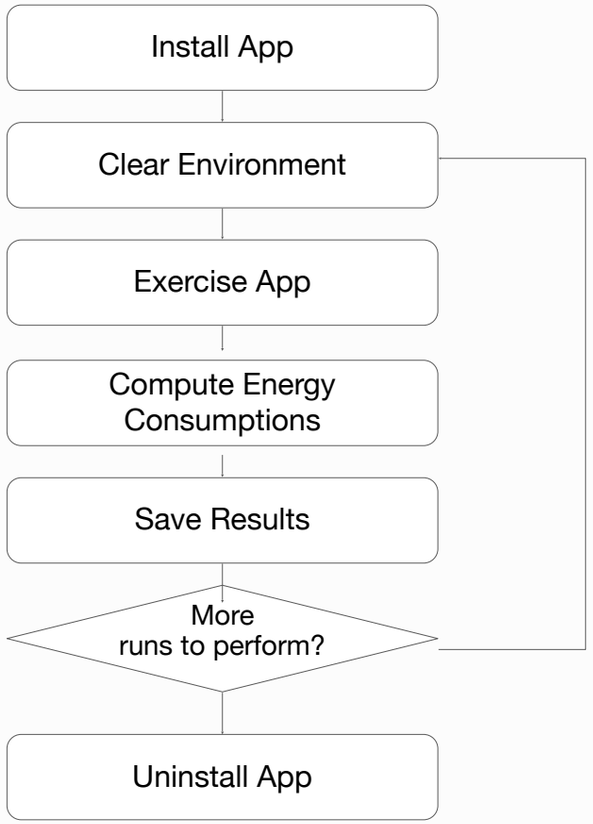


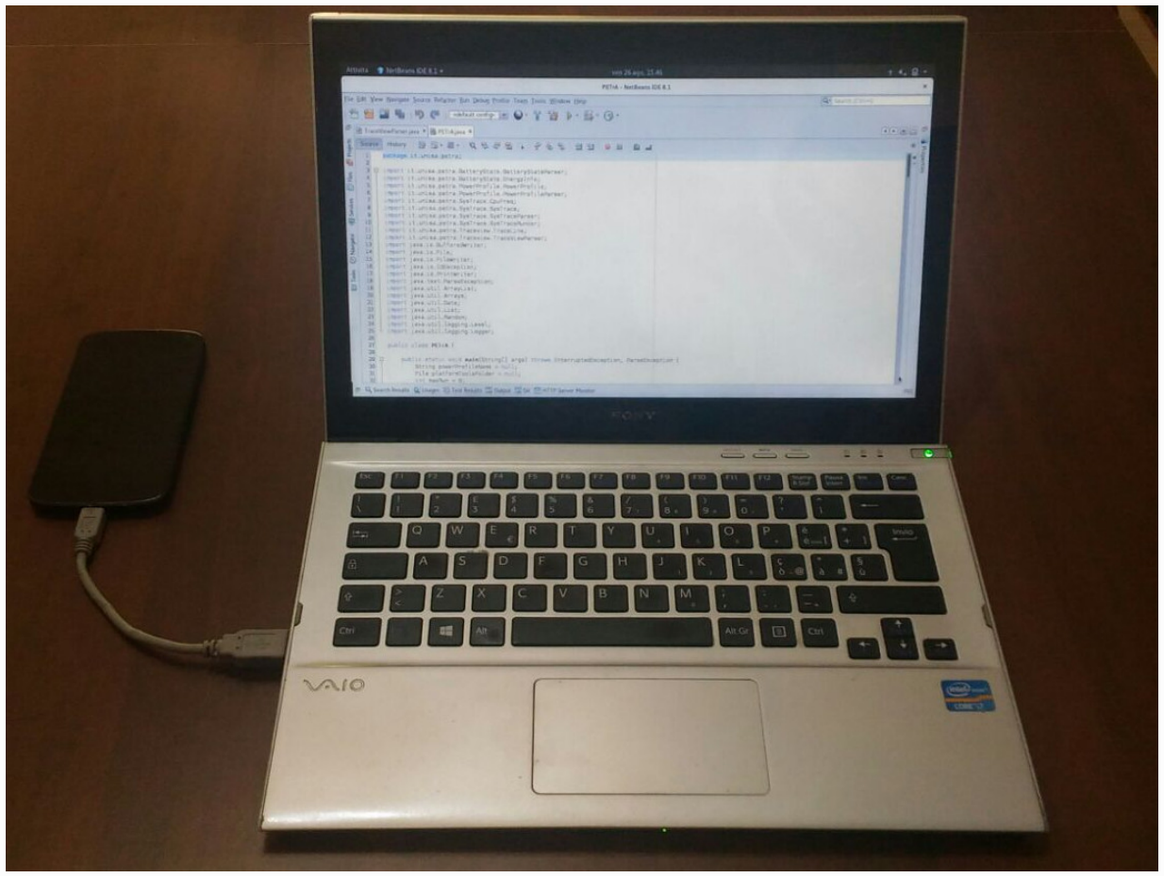
Software-based Tools: PETrA Power Estimation Tool for Android

PETrA is a tool based on the Project Volta self-modeling paradigm, which operates at the method level of granularity.

It minimizes the Hawthorne effect, and it is strongly integrated with Android.

It does not require any specialized HW, but a phone is needed.



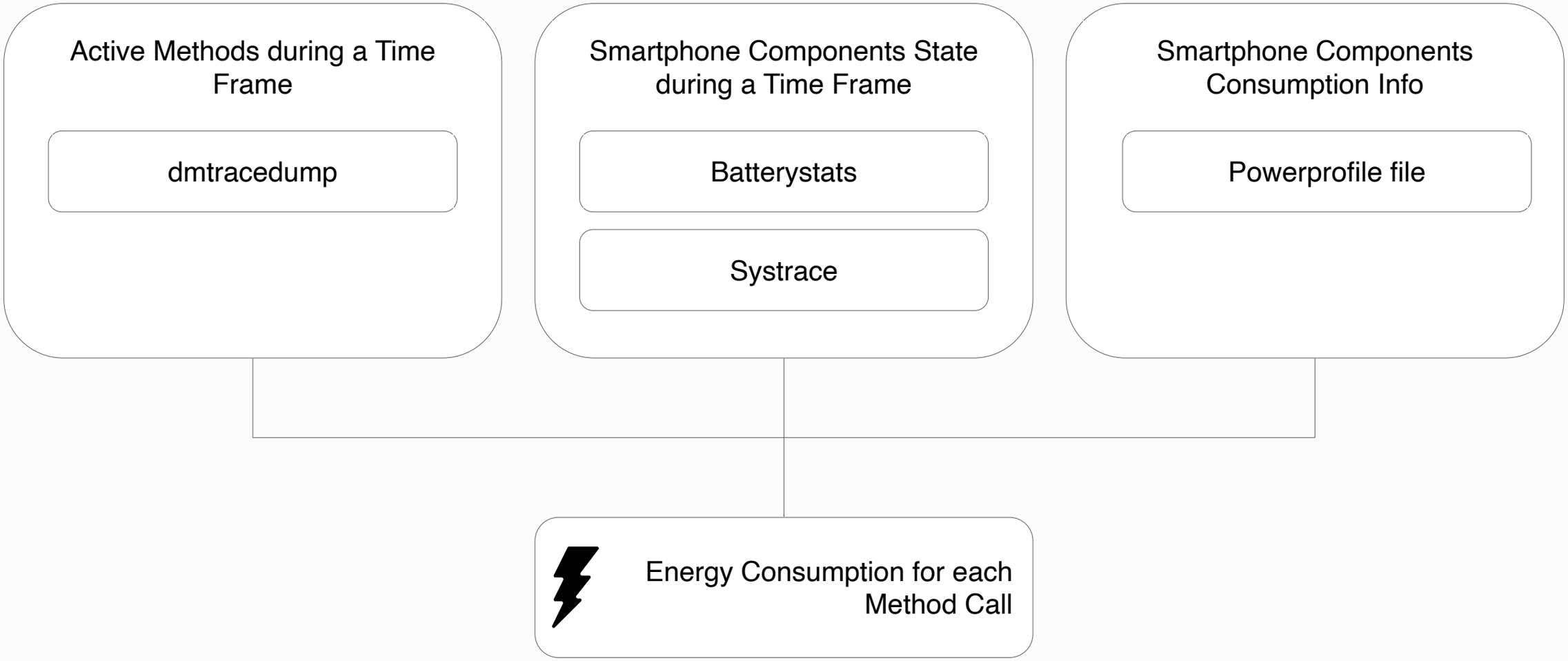


Dong, Mian, and Lin Zhong. "Self-constructive high-rate system energy modeling for battery-powered mobile systems." Proceedings of the 9th international conference on Mobile systems, applications, and services. 2011.

Di Nucci, Dario, et al. "Software-based energy profiling of android apps: Simple, efficient and reliable?." 2017 IEEE 24th international conference on software analysis, evolution and reengineering (SANER). IEEE, 2017.

https://www.youtube.com/watch?v=KzSKIpJepUw
https://github.com/dardin88/PETrA

Software-based Tools: PETrA Energy Profile Computation



Di Nucci, Dario, et al. "Software-based energy profiling of android apps: Simple, efficient and reliable?."
2017 IEEE 24th international conference on software analysis, evolution and reengineering (SANER). IEEE, 2017.

https://github.com/dardin88/PETrA

sesa lab
SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Measuring Energy Consumption



Slides courtesy of Dario Di Nucci | 2026 - 2027